

# Spring Framework: основы IoC и Dependency Injection

JAVA SPRING

НЕДЕЛЯ 1

# Сурен Калайчьян

Senior Developer | Ozon Tech

---

## Карьерный путь:

- Ozon Tech,
- MTC Web Services.

## Достижения:

- методист и лид курса Java в МФТИ ВШПИ и ЦУ, автор образовательных курсов;
- программный директор бакалавриата Ozon Tech;
- спикер на IT-конференциях и амбассадор года в MTC (2024).





# Александр Бобряков

TechLead | MTC Web Services

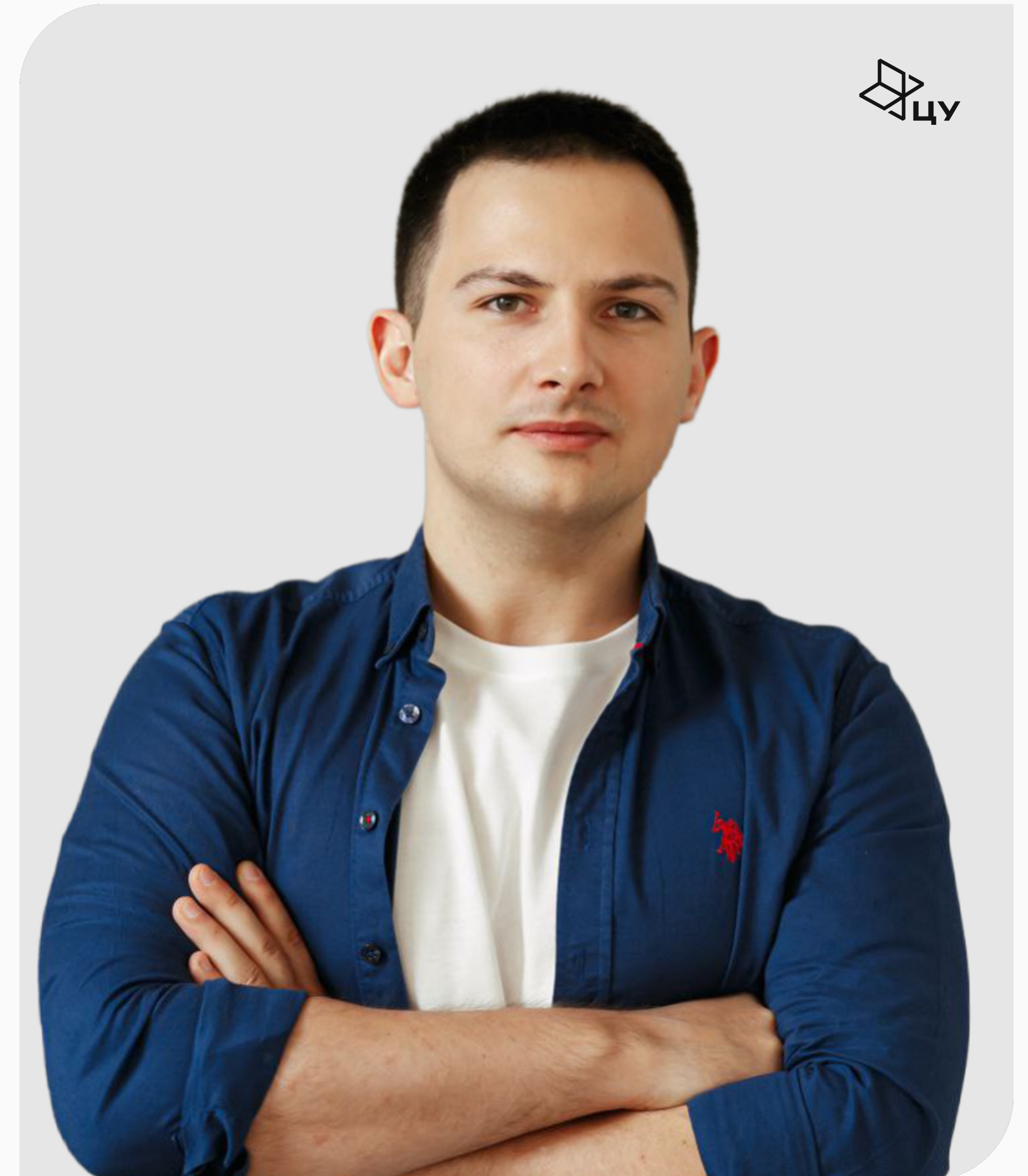
---

## Карьерный путь:

- MTC Web Services,
- «Сбер»,
- «Крок».

## Достижения:

- спикер на IT-конференциях, автор статей на «Хабр»;
- преподаватель в МФТИ и ЦУ;
- лид Java-направления в образовательных проектах MWS True Tech;
- сертифицированный эксперт по проведению собеседований на роли разработчиков и лидов;
- создатель матриц компетенций разработчиков.





# Дмитрий Бобряков

Senior developer | MTC Web Services

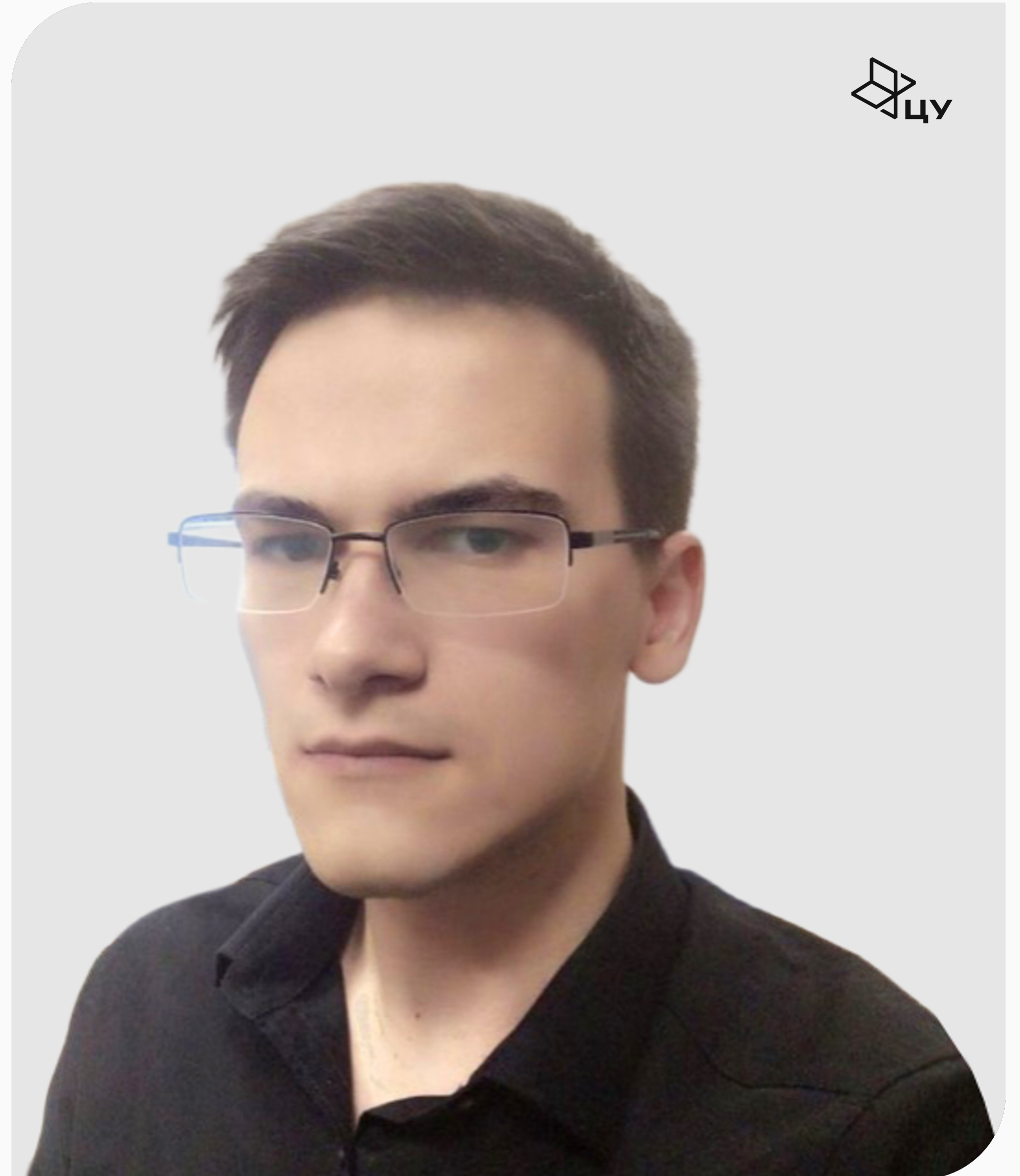
---

## Карьерный путь:

- MTC Web Services,
- «Альфа-банк»,
- NetCracker.

## Достижения:

- преподаватель в МФТИ и ЦУ,
- Senior-разработчик в BigData Streaming,
- Java-ментор в MWS.





# Что будет в этом курсе



- 01 Экосистема Spring. Spring Framework. Spring Boot
- 02 Работа с бинами. Жизненный цикл бинов
- 03 Стереотипные аннотации. Конфигурирование
- 04 RESTful API. Spring MVC. Обработка запросов
- 05 RESTful API. Spring MVC. Формирование ответов
- 06 Работа с БД. Spring JDBC. JdbcTemplate
- 07 Работа с БД. Spring Data JPA. Hibernate
- 08 Продвинутые возможности Spring Data JPA
- 09 Spring Security
- 10 Выполнение запросов. RestTemplate. HttpClient
- 11 Тестирование Spring-приложений
- 12 Профилирование и мониторинг Spring-приложений
- 13 Реактивный Spring. Spring WebFlux
- 14 Event-driven architecture. Работа с Kafka
- 15 Асинхронная работа с базами данных

# Система оценивания



| Активность       | Вес | Количество | Описание                                  |
|------------------|-----|------------|-------------------------------------------|
| Домашние задания | 70% | 5          | Задачи по темам недели                    |
| Экзамен          | 30% | 1          | Устная теоретическая и практическая части |

## Блокеры:

- Устная защита трёх домашних заданий.
- Экзамен на 4 балла и выше.

## ДЗ с устными защитами:

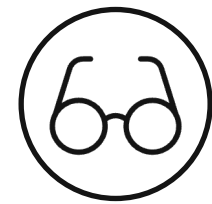
- ДЗ\_2 – Неделя 5
- ДЗ\_3 – Неделя 8
- ДЗ\_4 – Неделя 10



# План лекции

- Что такое фреймворк и библиотека
- Знакомство с Spring Framework
- Проблемы, которые решает Spring
- Dependency Injection и его отсутствие
- Inversion of Control (IoC)
- Способы внедрения зависимостей
- ApplicationContext
- Понятие бина
- Аннотации Spring
- Инструменты для работы с бинами
- FAQ

# Фреймворк vs библиотека

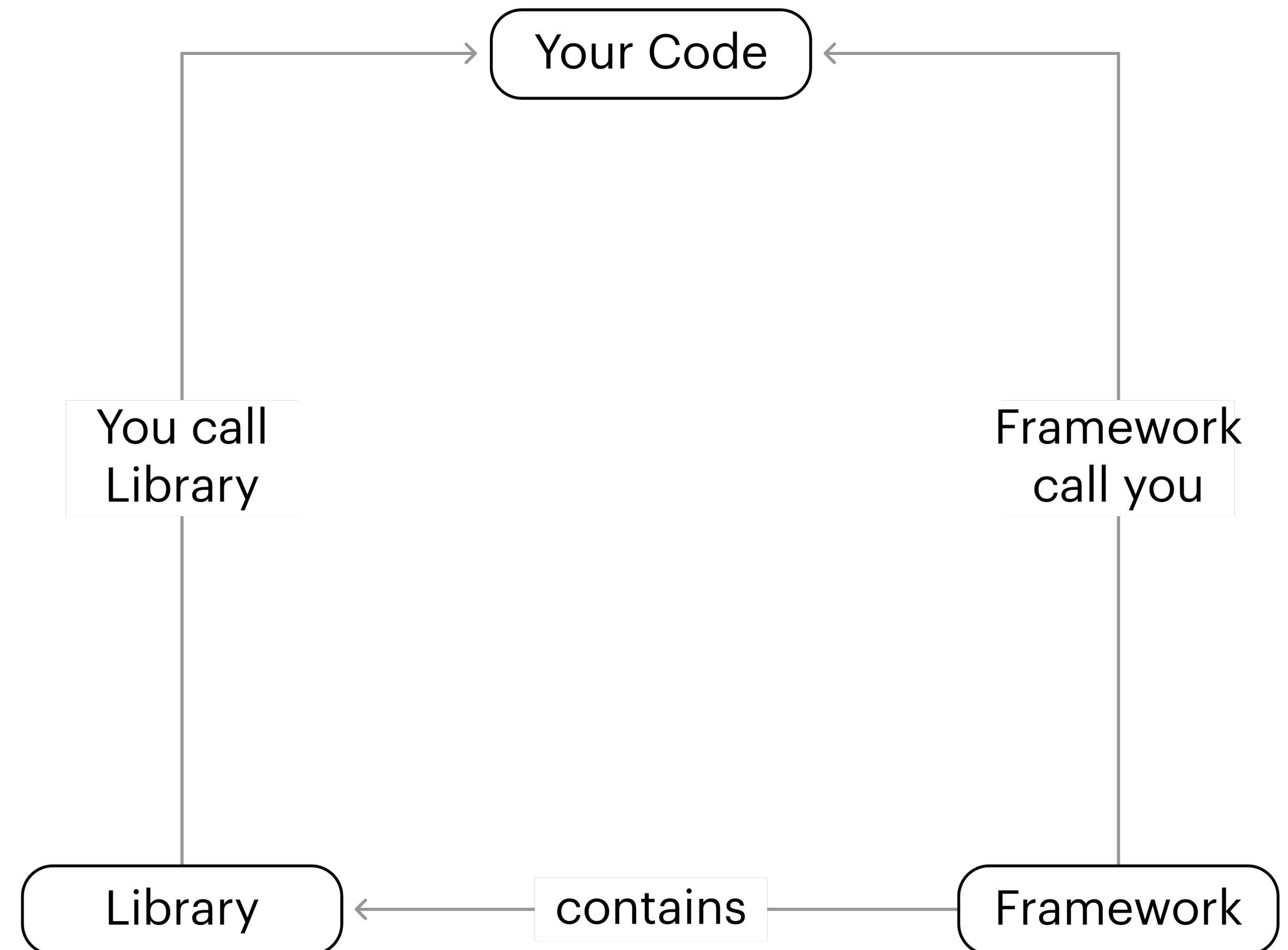


## Ключевое отличие

- Библиотека: вы вызываете функции.
- Фреймворк: фреймворк вызывает ваш код.

## Аналогия

- Библиотека = инструменты в ящике. Вы решаете, когда использовать молоток.
- Фреймворк = готовый каркас дома. Вы заполняете готовые места.



# Пример кода



...

```
// Библиотека: ВЫ управляете
```

```
StringUtils.isEmpty("text");
```

```
// Фреймворк: ФРЕЙМВОРК управляет
```

```
@GetMapping("/users")
```

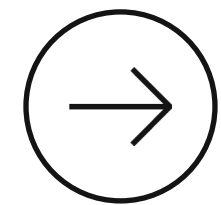
```
public String getUsers() {
```

```
    return "users";    // Spring вызывает этот метод
```

```
}
```



# Что такое Spring Framework?



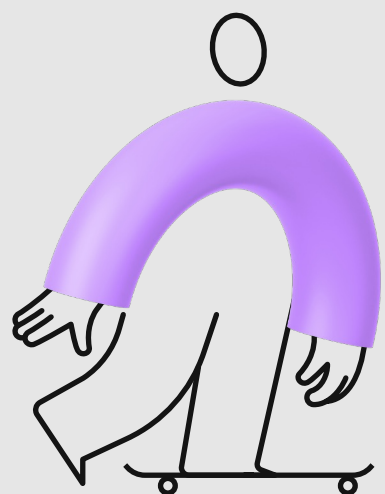
Модульный фреймворк для корпоративных Java-приложений

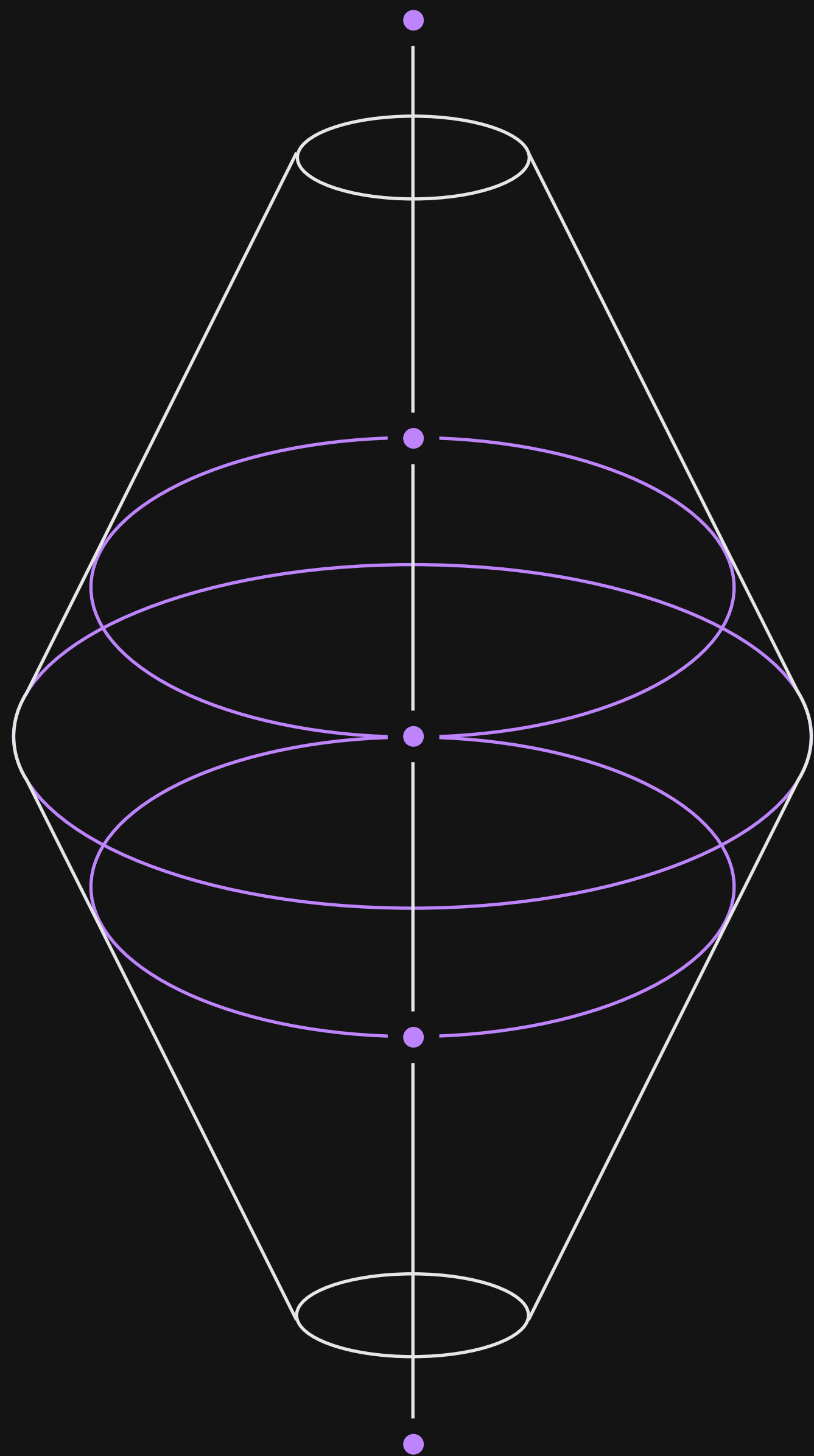


## История

- 2002: книга Рода Джонсона
- 2003: первый релиз Spring
- Эволюция: от EJB к легковесным POJO
- Стандарт де-факто в Java-экосистеме

- Полнофункциональный модульный фреймворк
- Для корпоративных Java-приложений
- Открытый исходный код
- Альтернатива тяжеловесным EJB





# Философия Spring



- Не изобретай велосипед
- Обратная связь с сообществом
- Backward compatibility
- Модульность

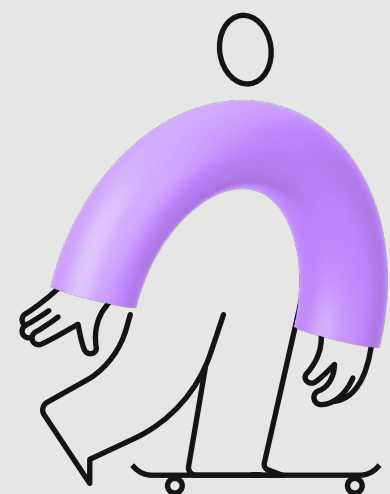
# Архитектура Spring



...

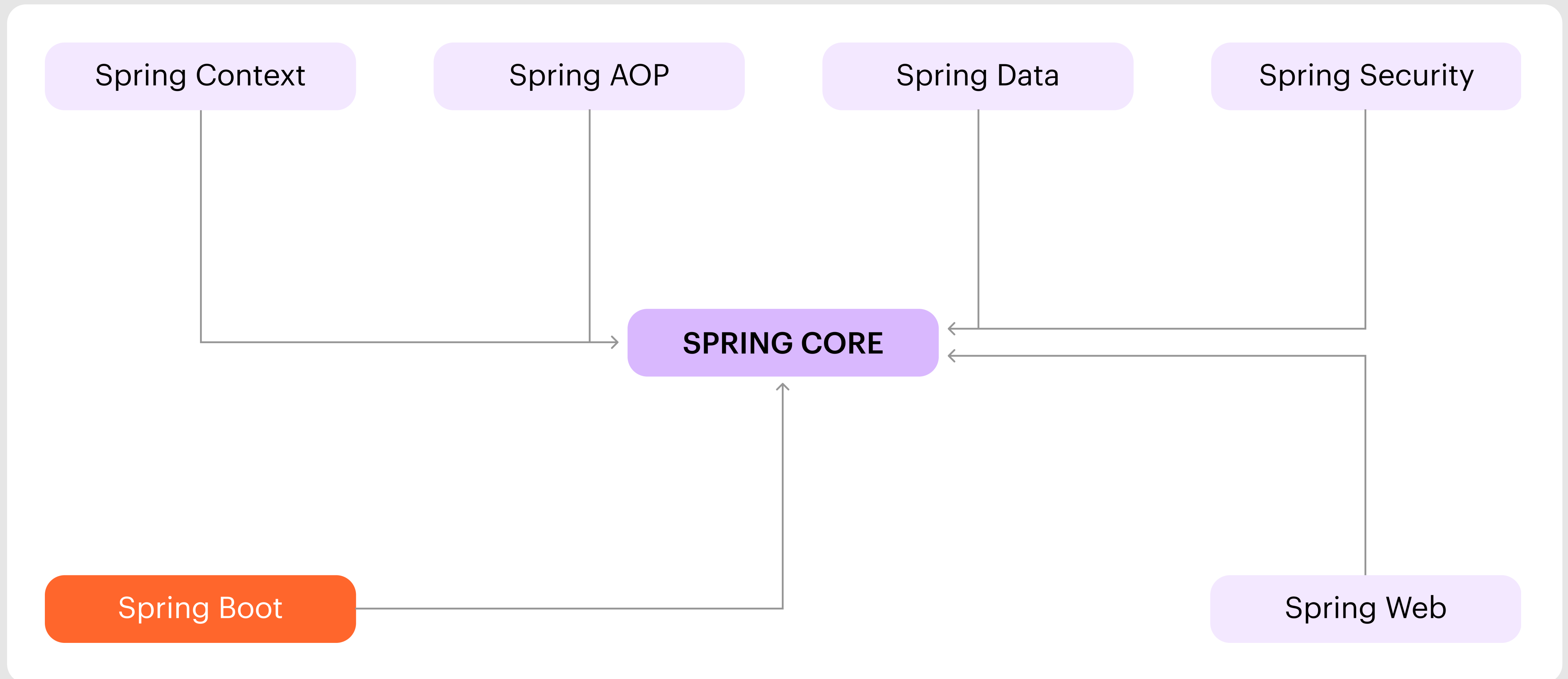
## Spring Framework

- Core Container (IoC, DI)
- AOP
- Data Access
  - JDBC
  - JPA/Hibernate
- Web (MVC, WebFlux)
  - Testing

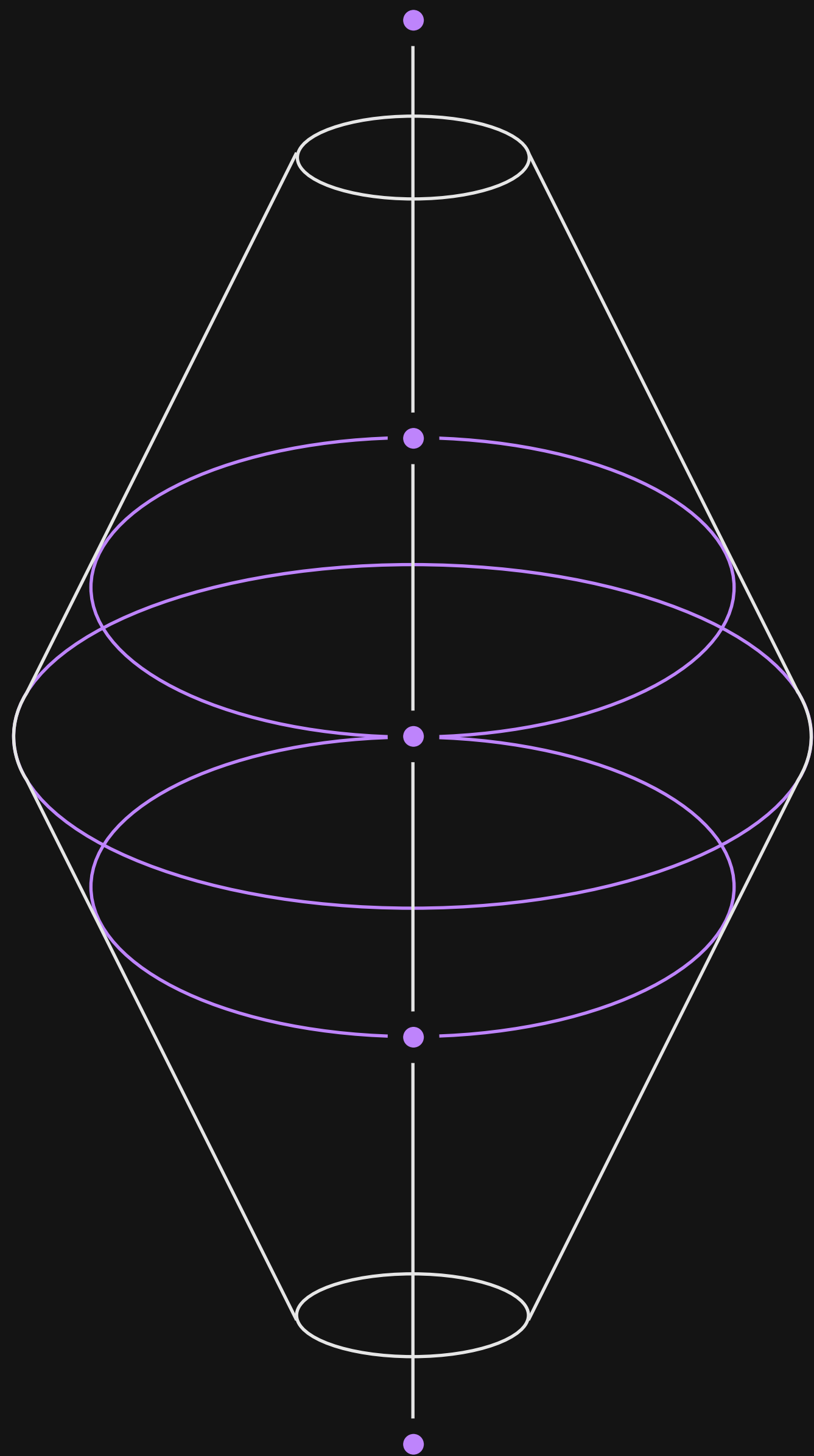




# Архитектура Spring







# Проблемы «до Spring»

- Сложность EJB
- Жёсткая связность (Tight Coupling)
- Ручное управление транзакциями
- Конфигурационная адская работа

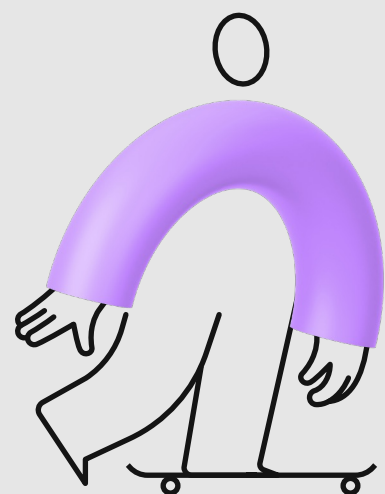
# Проблемы «до Spring»



## 01 Сложность EJB

...

```
// EJB 2.x - 100+ строк кода
public class UserBean implements EntityBean {
    // Десятки обязательных методов
}
```



# Проблемы «до Spring»

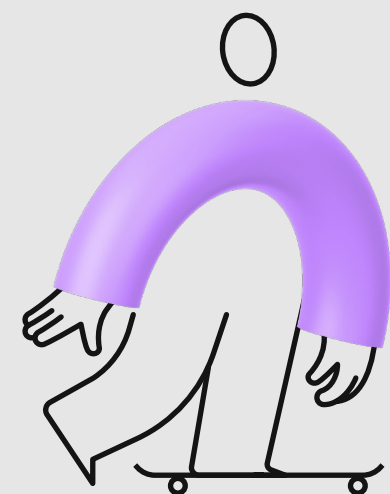


02

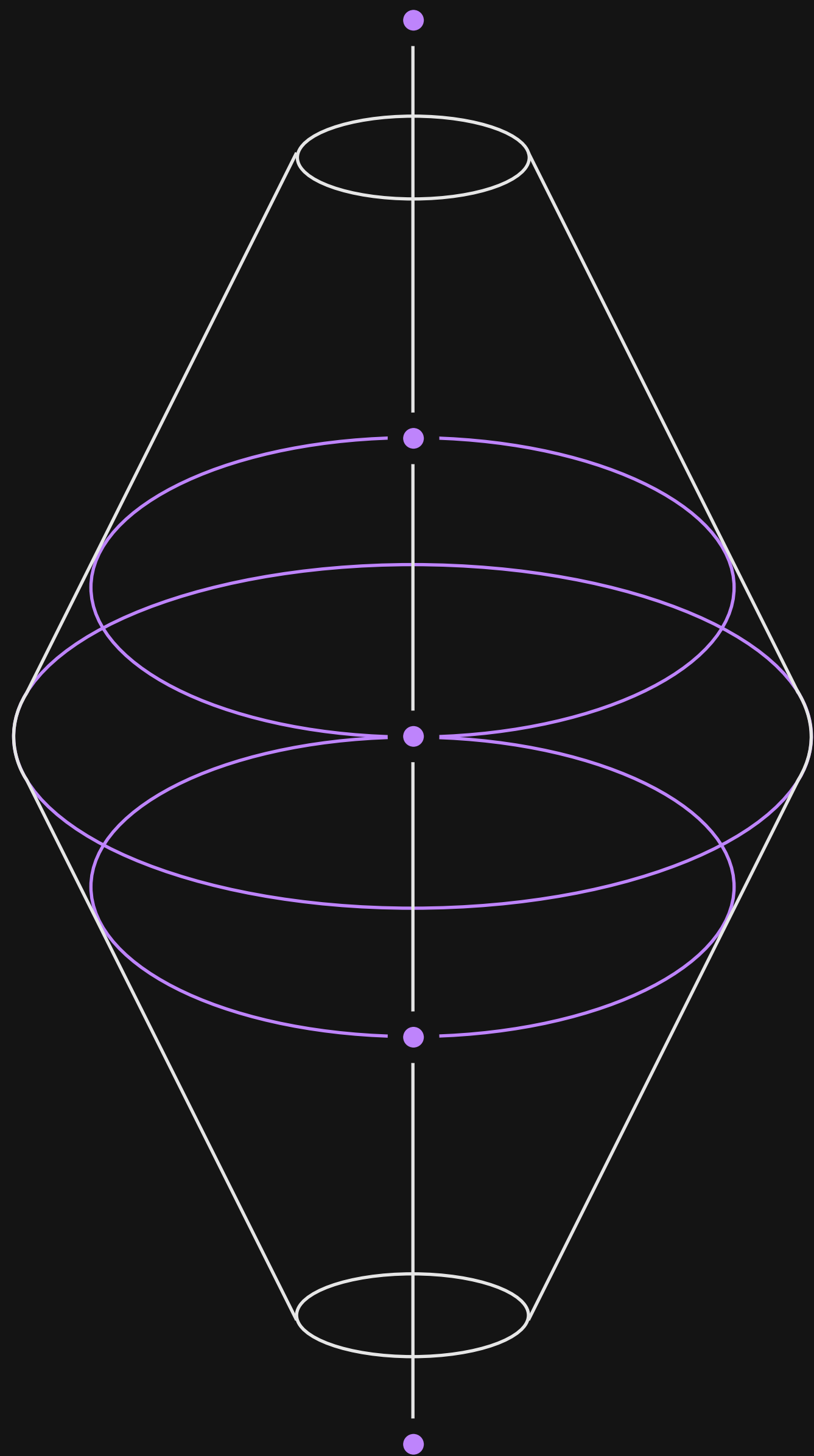
Жёсткая связность

...

```
public class UserService {  
    private UserRepository repo = new UserRepositoryImpl();  
    // Невозможно заменить реализацию  
}
```





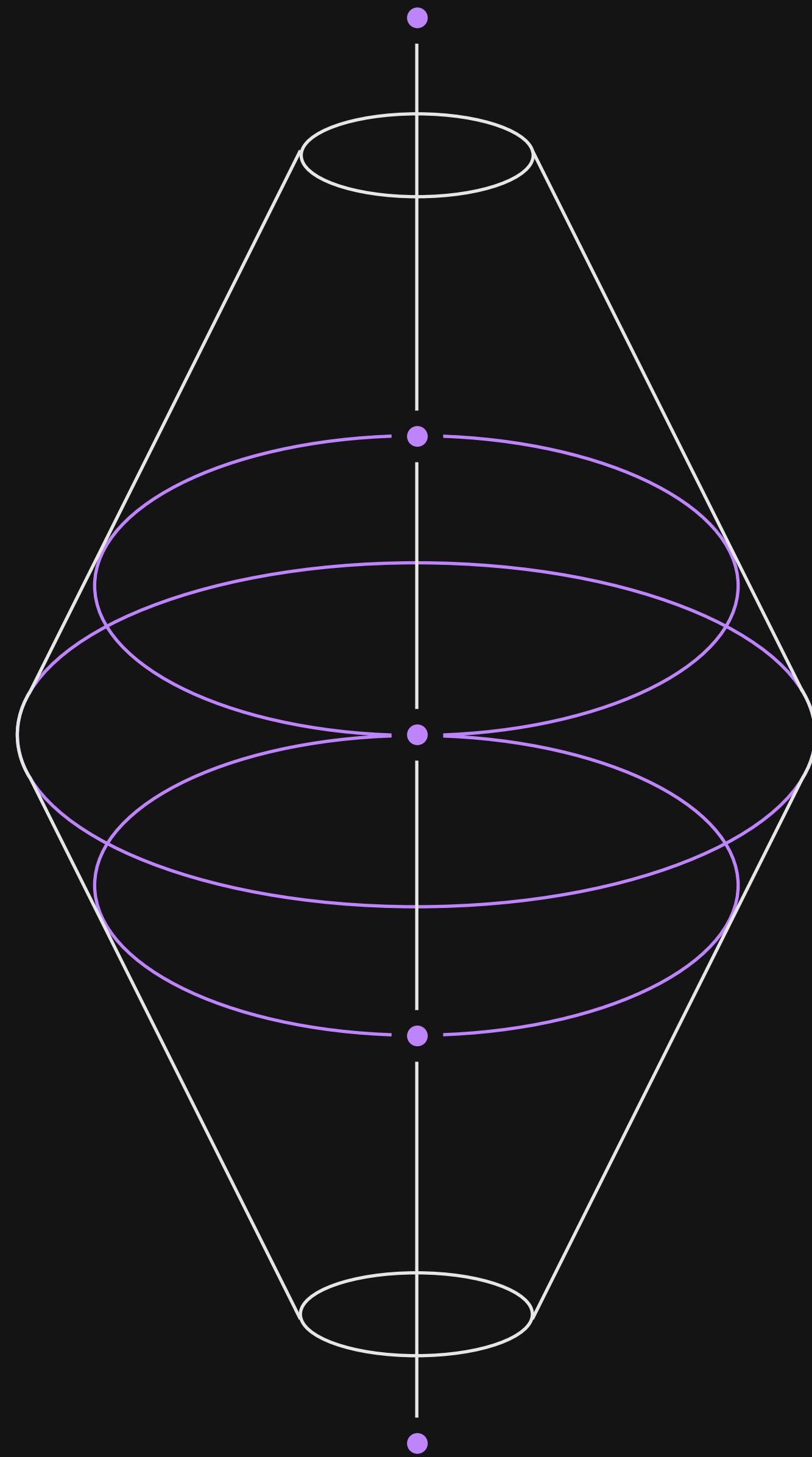


# Как Spring решает проблемы?



- Легковесность через POJO
- Внедрение зависимостей
- Декларативное управление транзакциями
- Модульность и интеграция

# Spring Boot — ЭВОЛЮЦИЯ



- Автоконфигурация
- Starter-зависимости
- Embedded-серверы
- Упрощение конфигурации



# Spring Boot



## Spring Boot Features

**01**  
Spring-Application

**02**  
Lazy Initialization

**03**  
Admin Features

**04**  
Security

**05**  
Logging

**06**  
Caching

**07**  
Kotlin Support

**08**  
Validation

**09**  
JSON

**10**  
Testing

**11**  
Task Execution and Scheduling

# Проблема отсутствия DI

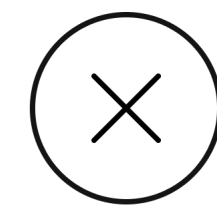


## Что такое Dependency Injection?

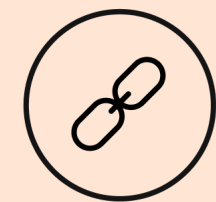
- Паттерн для управления зависимостями
- Зависимости предоставляются извне
- Устраняет жёсткие связи

## Проблемы

- Невозможность тестирования
- Сложность замены реализации
- Нарушение SRP
- Проблемы с жизненным циклом



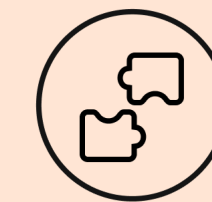
## Common Pitfalls Without Dependency Injection



Tight Coupling



Poor Maintainability



No Reusability

# Решение через DI

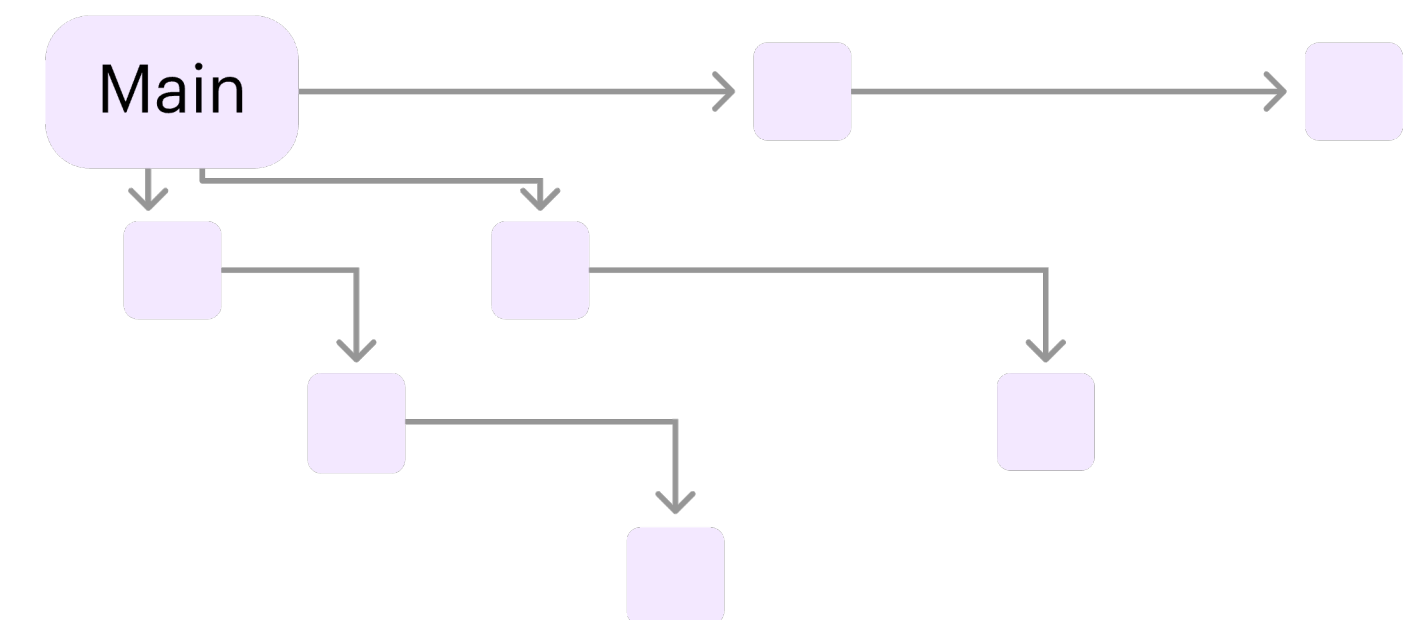


...

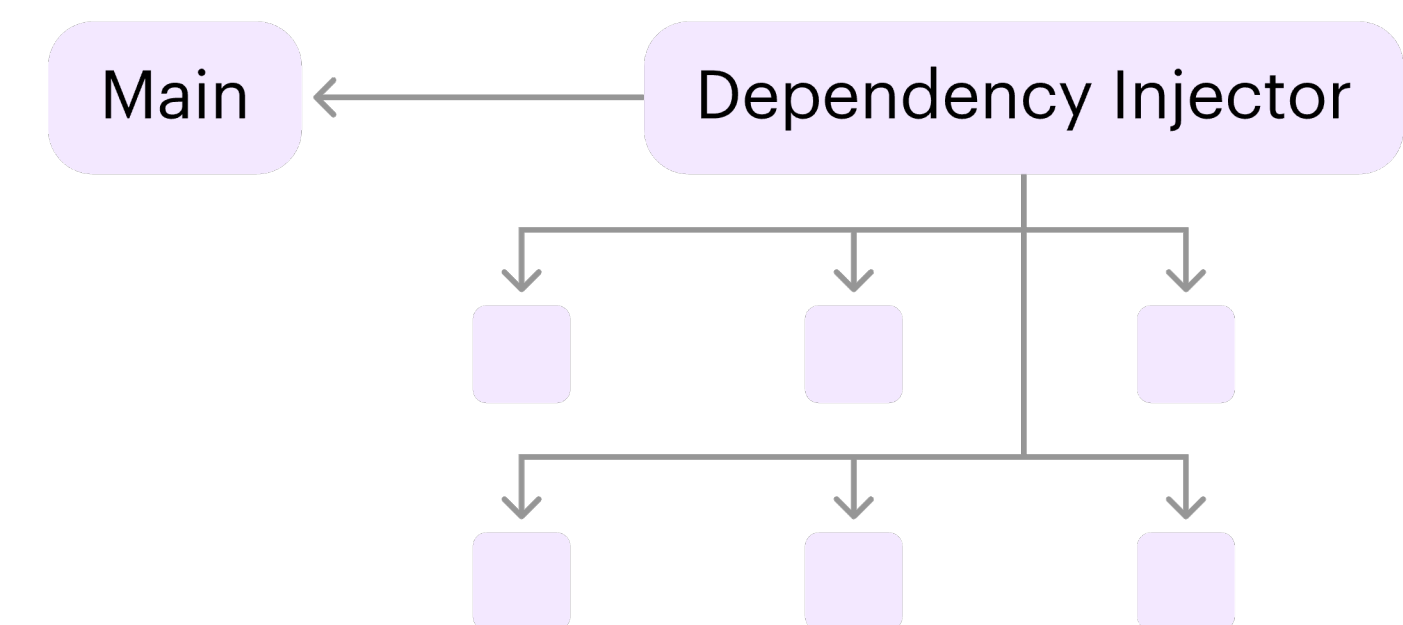
```
public class Service {  
    private Repository repo = new RepositoryImpl();  
    // Тестирование? Замена? Управление?  
}
```

```
public class Service {  
    private final Repository repo;  
  
    // Зависимость приходит извне  
    public Service(Repository repo) {  
        this.repo = repo;  
    }  
}
```

## Traditional



## Dependency Injected

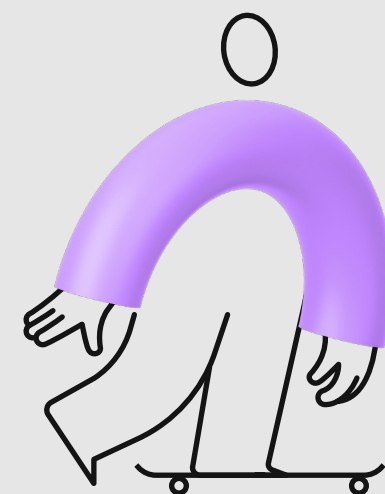
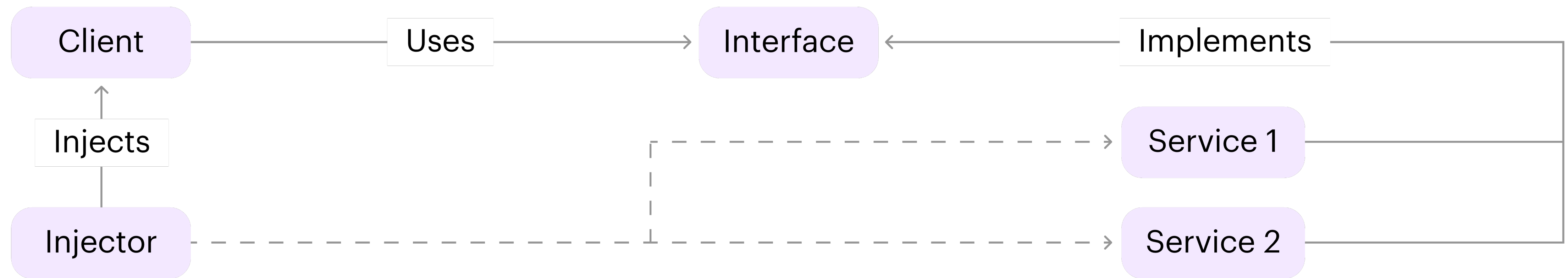




# Решение через DI



## Key Roles of Dependency Injection



# Dependency Injection



3 способа внедрения

01 Field Injection (не рекомендуется)

• • •

```
@Autowired  
private Repository repo;
```

02 Field Injection (не рекомендуется)

• • •

```
@Autowired  
public void setRepo(Repository  
repo) {  
    this.repo = repo;  
}
```

# Dependency Injection



## 03 Constructor Injection (рекомендуется)

...

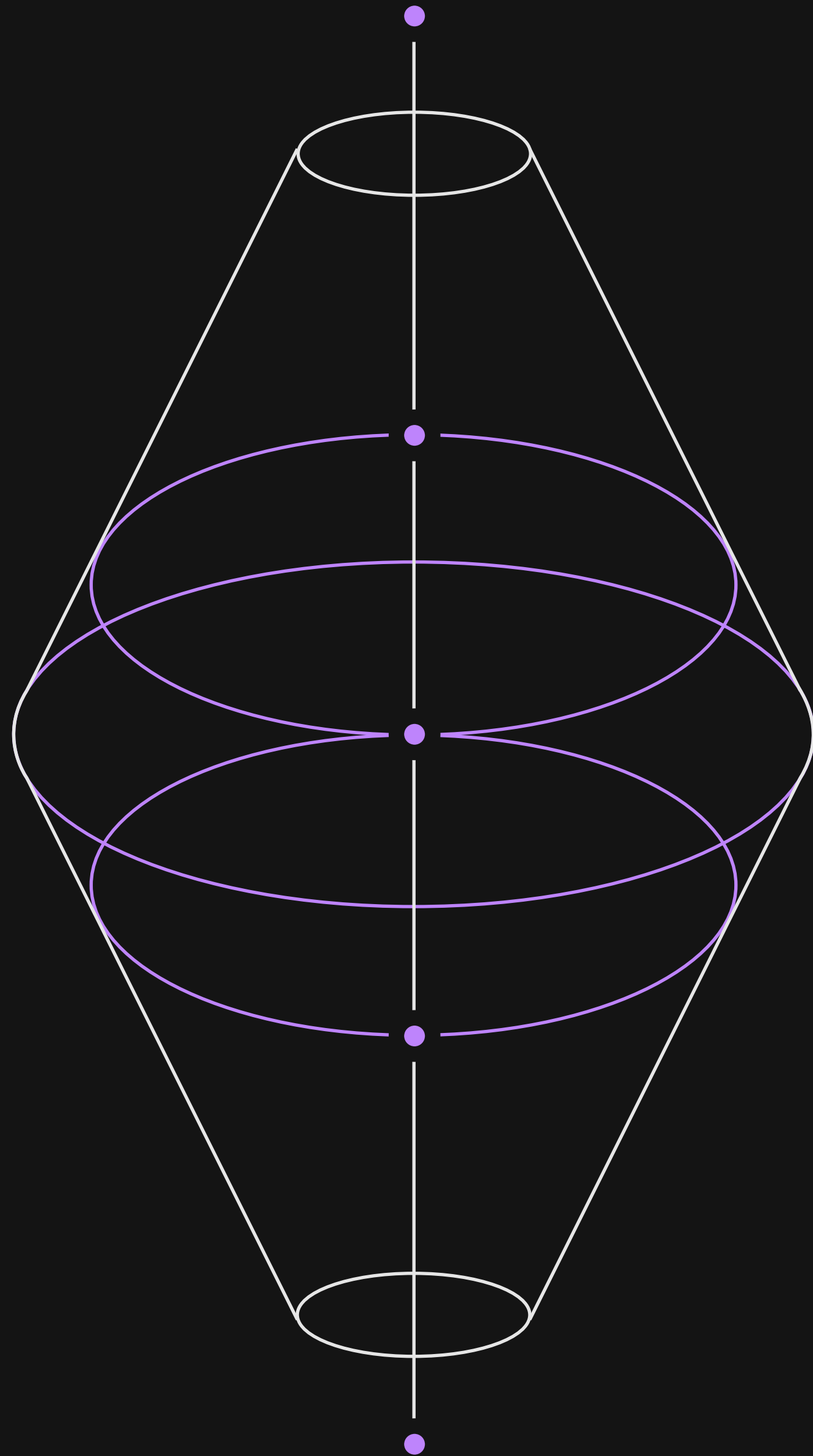
```
private final Repository repo;  
public Service(Repository repo)  
{  
    this.repo = repo;  
}
```



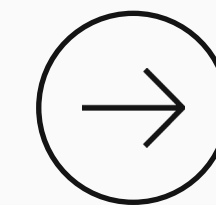
# Constructor Injection

## Преимущества

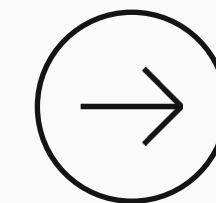
- Неизменяемость (поля final)
- Гарантированная инициализация
- Лучшая тестируемость
- Обнаружение циклических зависимостей



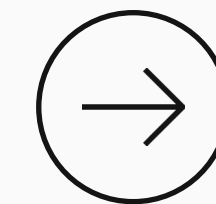
# Рекомендации Spring Team



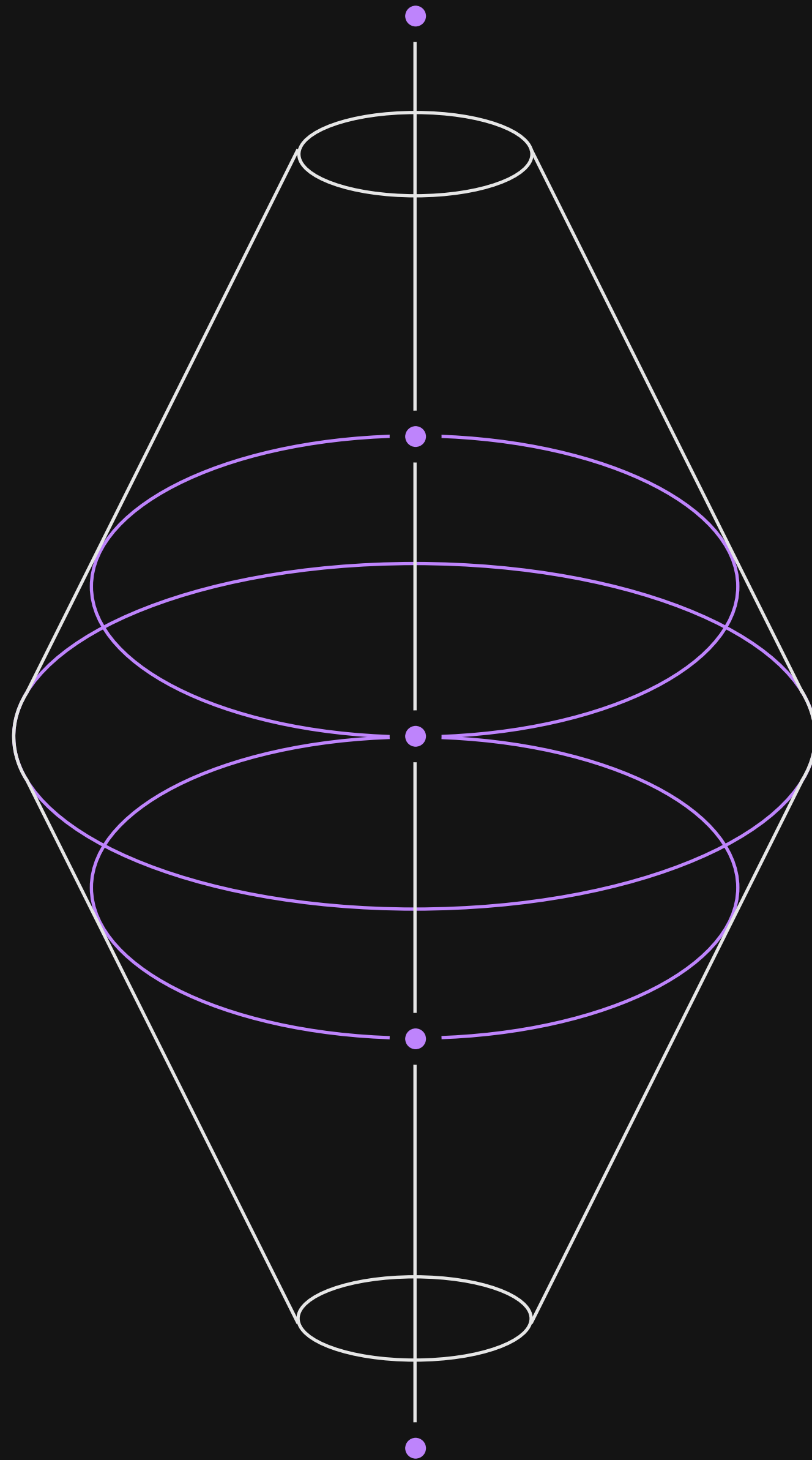
Constructor Injection  
для обязательных зависимостей



Setter Injection для optional



Field Injection — избегать

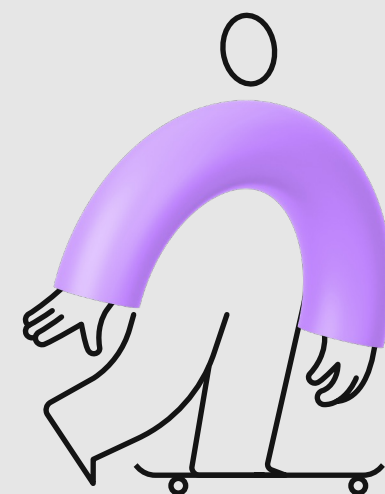


# Пример тестирования



...

```
@Test
void test() {
    Repository mockRepo = mock(Repository.class);
    Service service = new Service(mockRepo); // Чистый unit-тест
}
```



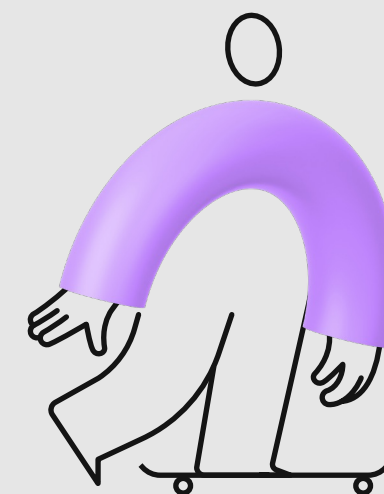


# Inversion of Control (IoC)



## ОПРЕДЕЛЕНИЕ

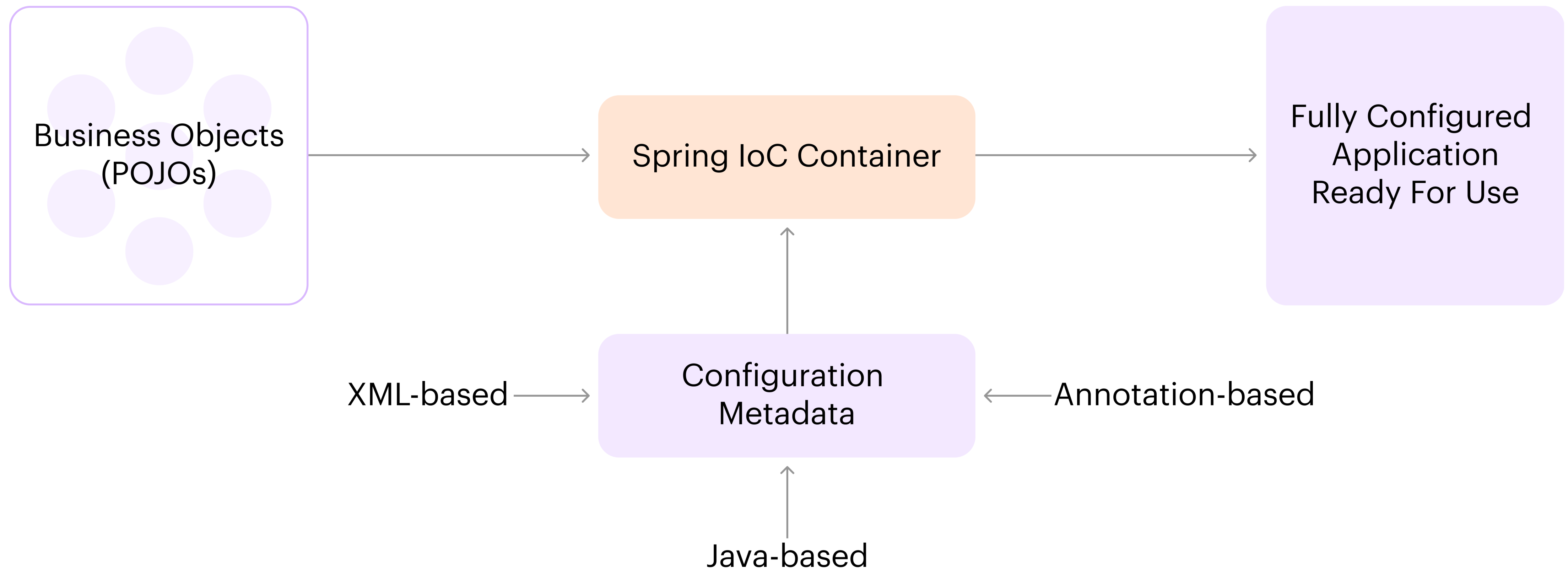
- Инверсия управления
- Контроль передаётся от программиста к фреймворку
- Spring реализует через контейнер



# Inversion of Control (IoC)



## Spring IoC Container



# Inversion of Control (IoC)



## ТРАДИЦИОННЫЙ ПОДХОД

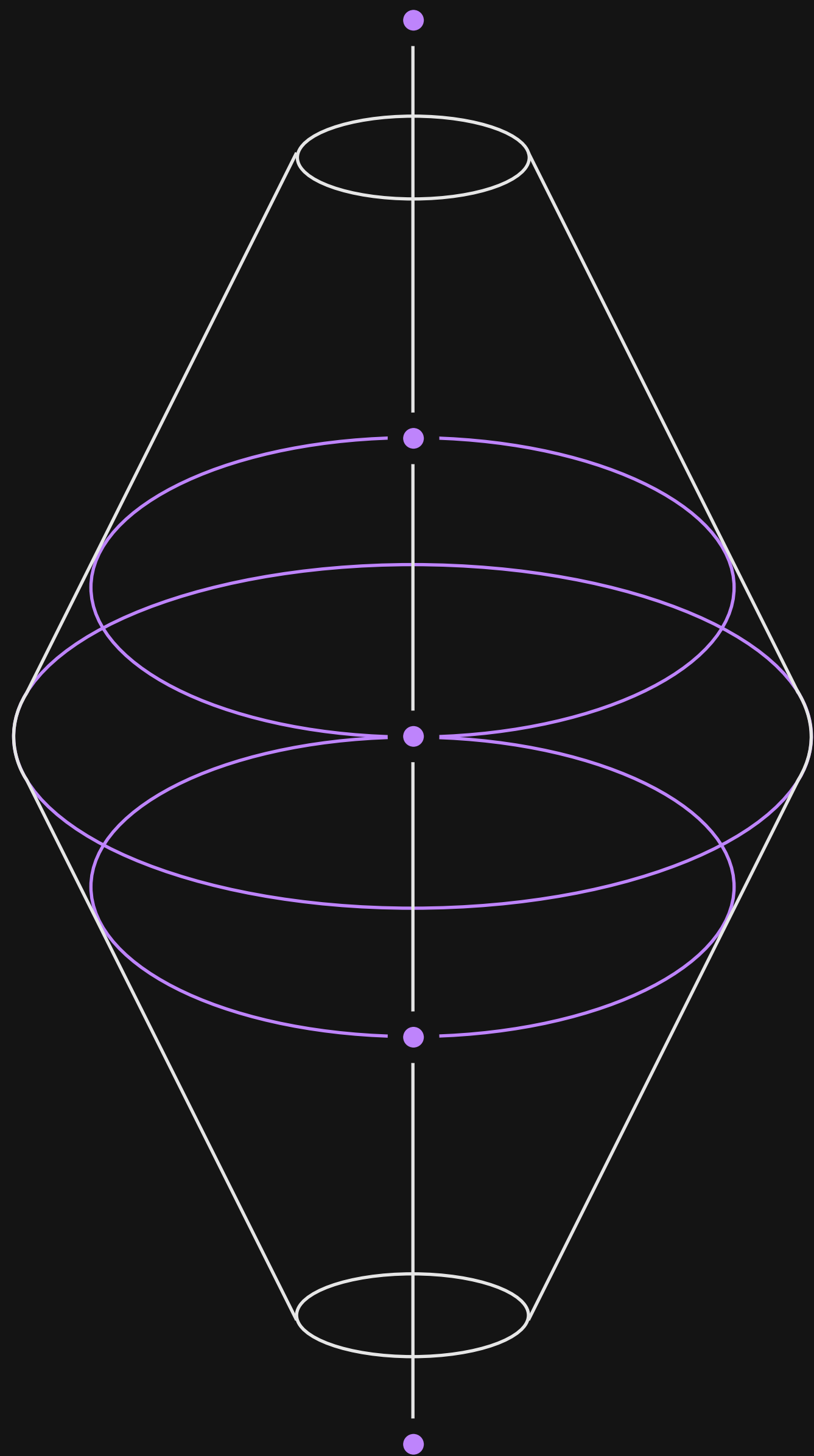
.Main → создаёт A → создаёт B → вызывает методы

## ИОС-ПОДХОД

.IoC Container → создаёт B → создаёт A → Main запрашивает A





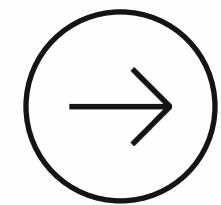


# Преимущества IoC



- Слабая связность
- Централизованная конфигурация
- Упрощение тестирования
- Управление жизненным циклом

# ApplicationContext



Центральный интерфейс Spring

## Что делает?

- Создаёт бины, управляет ими и их жизненным циклом
- Внедряет зависимости
- Управляет жизненным циклом
- Публикует события
- Центральный интерфейс IoC-контейнера
- Предоставляет доступ к ресурсам

# Иерархия контекстов



...

BeanFactory



ApplicationContext



ConfigurableApplicationContext



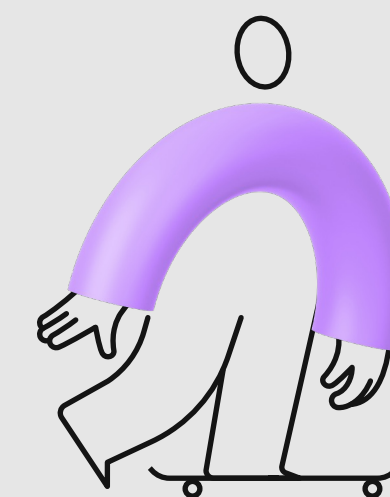
ClassPathXmlApplicationContext



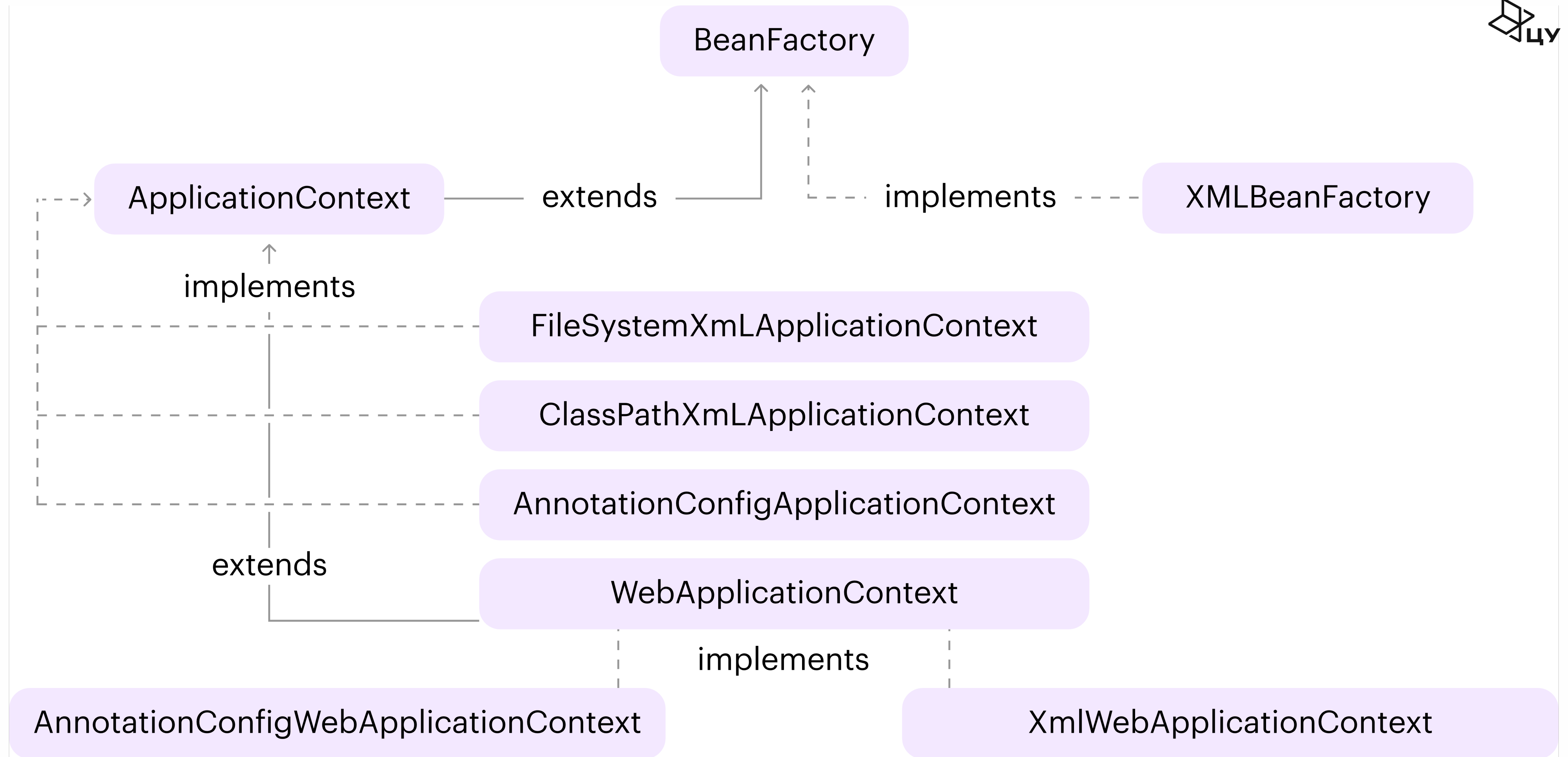
AnnotationConfigApplicationContext



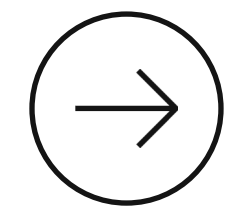
...





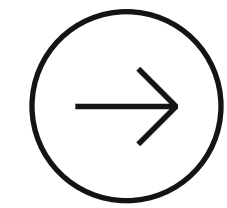


# Создание контекста



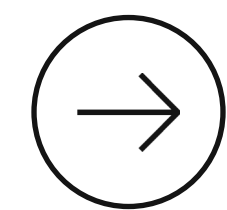
**// XML-конфигурация**

```
new ClassPathXmlApplicationContext("config.xml");
```



**// Java-конфигурация**

```
new AnnotationConfigApplicationContext(AppConfig.class);
```



**// Spring Boot**

```
SpringApplication.run(App.class, args);
```

# Что такое бин (Bean)?

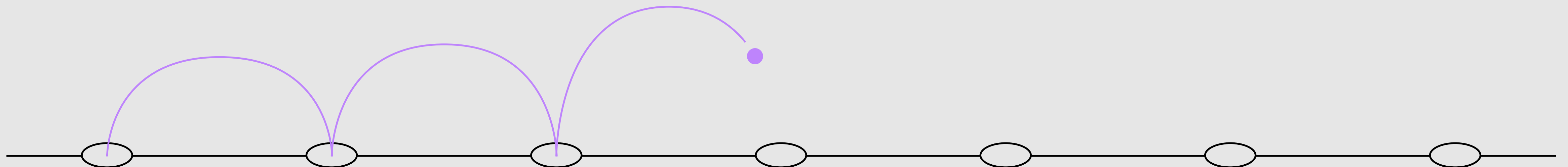


## Объект,

- который создаётся Spring-контейнером,
- имеет уникальное имя,
- управляется контейнером,
- может иметь зависимости.

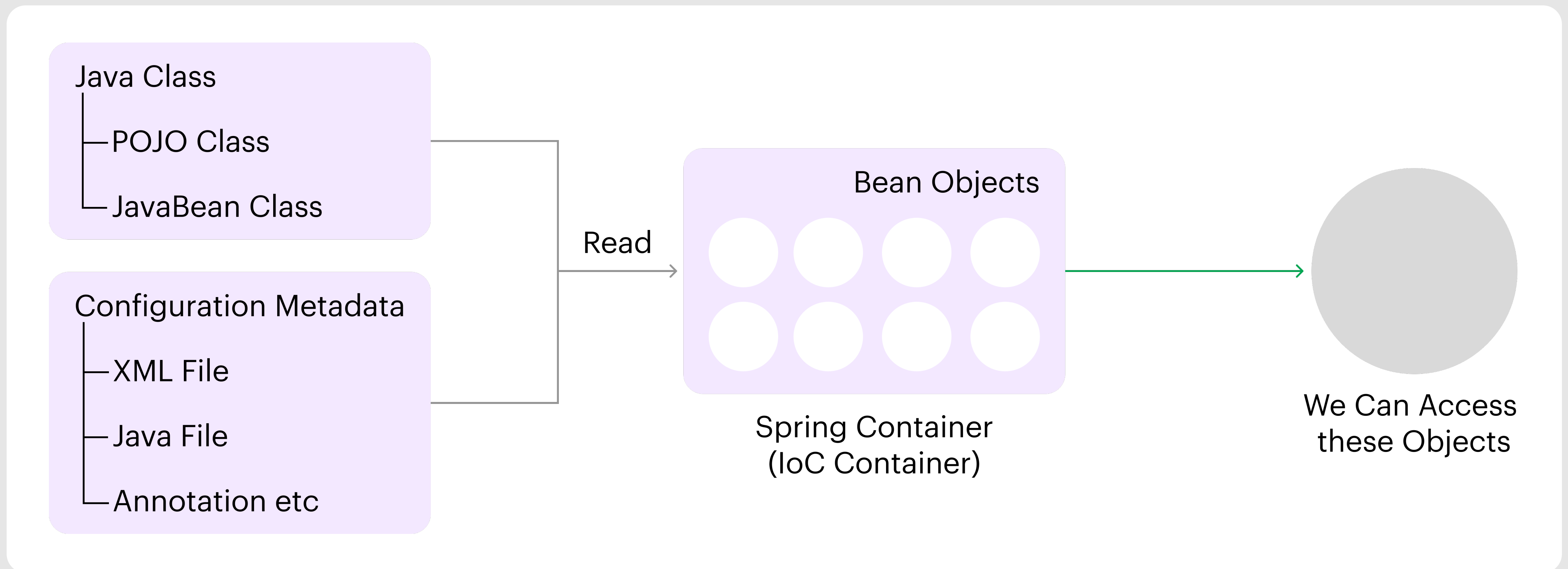
## Способы создания:

- XML-конфигурация (legacy);
- аннотации (@Component, @Service);
- Java Config (@Configuration, @Bean).





# Что такое бин (Bean)?



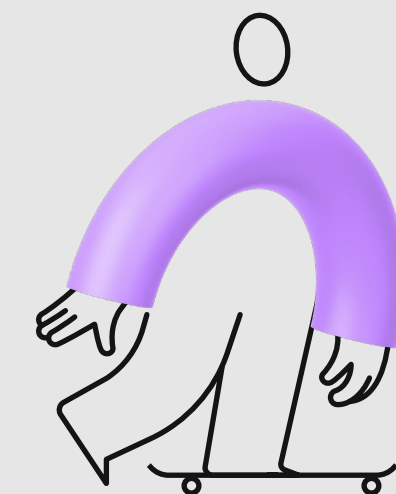


# Создание бинов: XML



...

```
<bean id="userService" class="com.example.UserService">  
  <property name="repository" ref="userRepository"/>  
</bean>
```



# Создание бинов: Java Config



...

```
@Configuration
public class AppConfig {
    @Bean
    public UserService userService() {
        return new UserService(userRepository());
    }

    @Bean
    public UserRepository userRepository() {
        return new UserRepositoryImpl();
    }
}
```

# Создание бинов через аннотации



...

```
@Service // Бизнес-логика
public class UserService {
    private final UserRepository repo;
    public UserService(UserRepository repo) {
        this.repo = repo;
    }
}

@Repository // Доступ к данным
public class UserRepositoryImpl implements UserRepository {
    // Автоматическое преобразование исключений
}

@RestController // REST API
public class UserController {
    @GetMapping("/users")
    public List<User> getUsers() { ... }
}
```



# Проблема: несколько бинов одного типа



...

```
@Component
public class EmailService implements NotificationService {}

@Component
public class SmsService implements NotificationService {}

@Service
public class UserService {
    @Autowired // ОШИБКА! Какой выбрать?
    private NotificationService service;
}
```

# @Qualifier



## РЕШЕНИЕ 1

• • •

```
@Component
@Qualifier("email")
public class EmailService implements NotificationService {}
```

```
@Component
@Qualifier("sms")
public class SmsService implements NotificationService {}
```

```
@Service
public class UserService {
    @Autowired
    @Qualifier("email") // Явное указание
    private NotificationService service;
}
```

# @Primary



## РЕШЕНИЕ 2

• • •

```
@Component
@Primary // Бин по умолчанию
public class EmailService implements NotificationService {}

@Component
public class SmsService implements NotificationService {}

@Service
public class UserService {
    @Autowired // Выберется EmailService
    private NotificationService service;
}
```



# Autowiring by Name



...

```
@Component // Имя бина: "emailNotificationService"
public class EmailNotificationService implements NotificationService {}

@Service
public class UserService {
    @Autowired
    // Имя поля совпадает с именем бина!
    private NotificationService emailNotificationService;
}
```

# Инъекция коллекций



...

```
@Component
public class NotificationDispatcher {
    // Все бины типа NotificationService
    @Autowired
    private List<NotificationService> allServices;

    // Map: имя бина → бин
    @Autowired
    private Map<String, NotificationService> serviceMap;

    // Только с определённым Qualifier
    @Autowired
    @Qualifier("email")
    private List<NotificationService> emailServices;
}
```

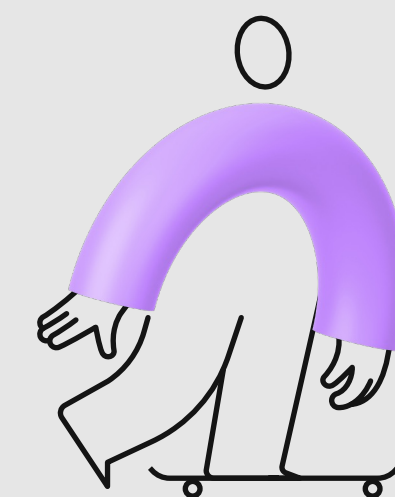


# Injection



## Порядок разрешения @Autowired

1. По типу → 1 бин? ✓
2. Несколько бинов? → Есть @Primary? ✓
3. Нет @Primary? → Есть @Qualifier? ✓
4. Нет @Qualifier? → По имени поля ✓
5. Иначе → NoUniqueBeanDefinitionException ✗



# Жизненный цикл бина

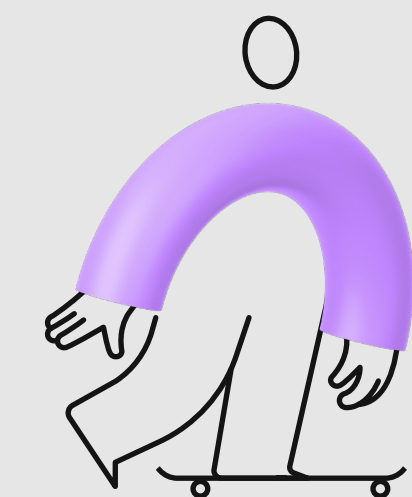


## Порядок инициализации

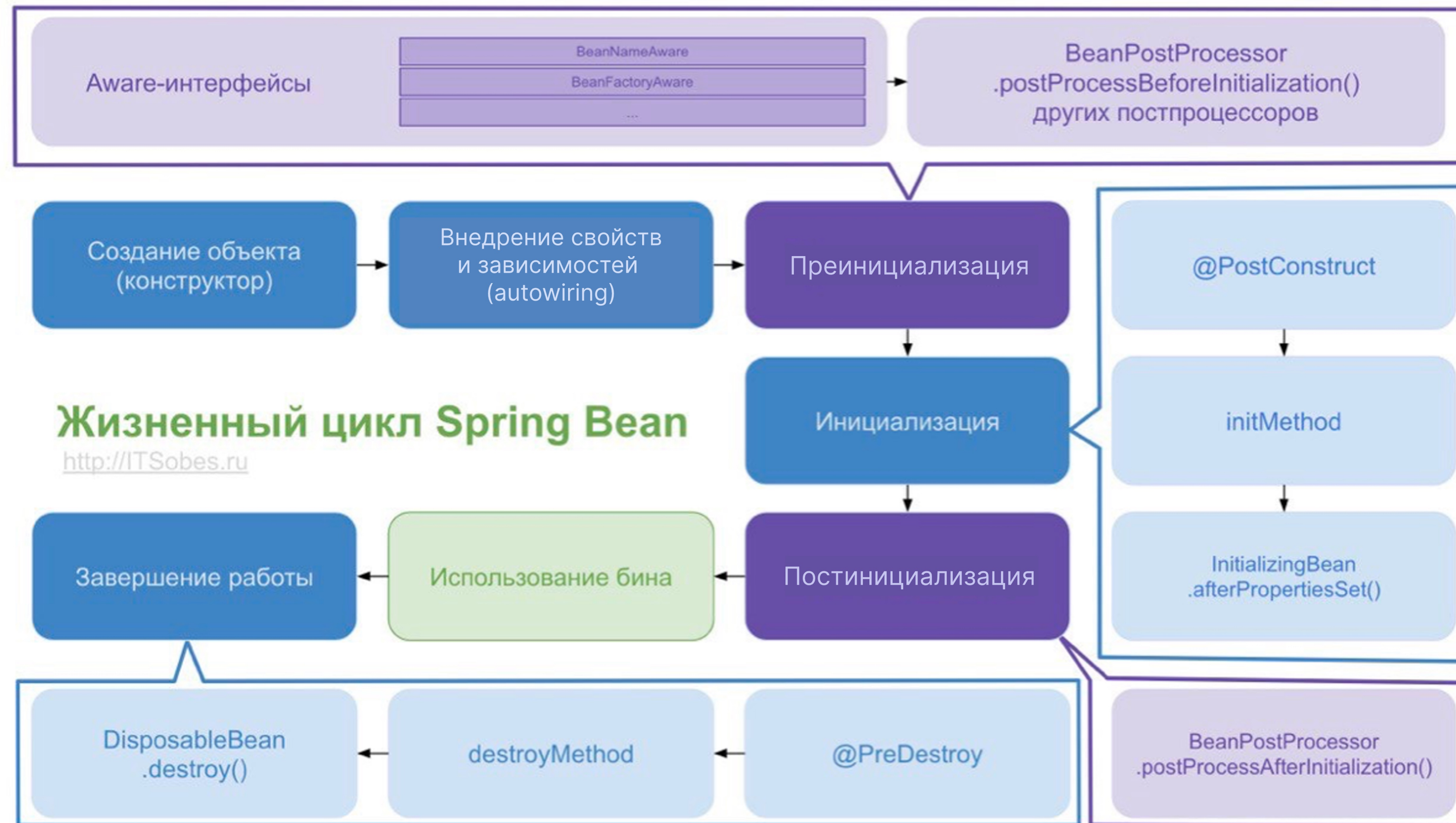
1. Конструктор
2. Внедрение зависимостей
3. @PostConstruct
4. InitializingBean.afterPropertiesSet()
5. Кастомный initMethod

## Порядок уничтожения

1. @PreDestroy
2. DisposableBean.destroy()
3. Кастомный destroyMethod



# Beans' lifecycle





# Дополнительно: управление бинами



## Annotations Used In Spring Bean

### Used In Spring Bean Properties

@Lazy

@Profile

@Scope

@Order

@Primary

@Conditional

@DependsOn

### Used In Spring Bean Injection

@Autowired

@Resource

@Inject

@Qualifier

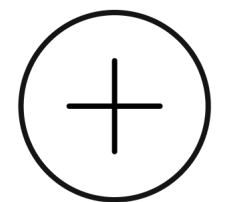
@Primary

@Primary vs. @Qualifier

### Used In Spring Bean State

@PostConstruct

@PreDestroy

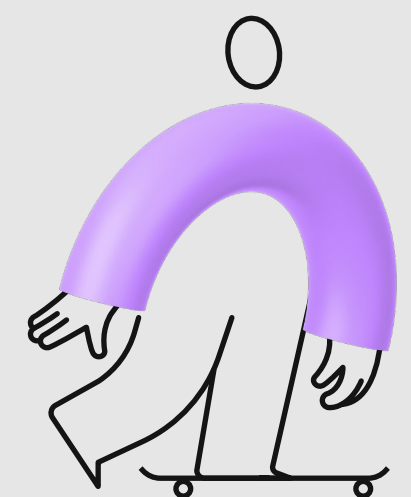


# Стереотипы бинов



## Аннотация @Component

- Базовый стереотип
- Помечает класс как Spring-компонент
- Имя бина = имя класса с маленькой буквы

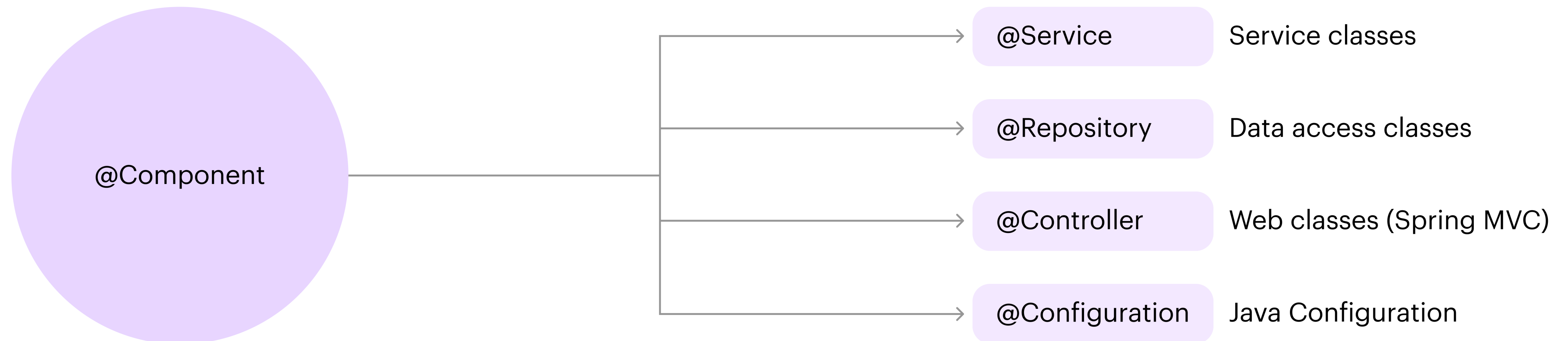




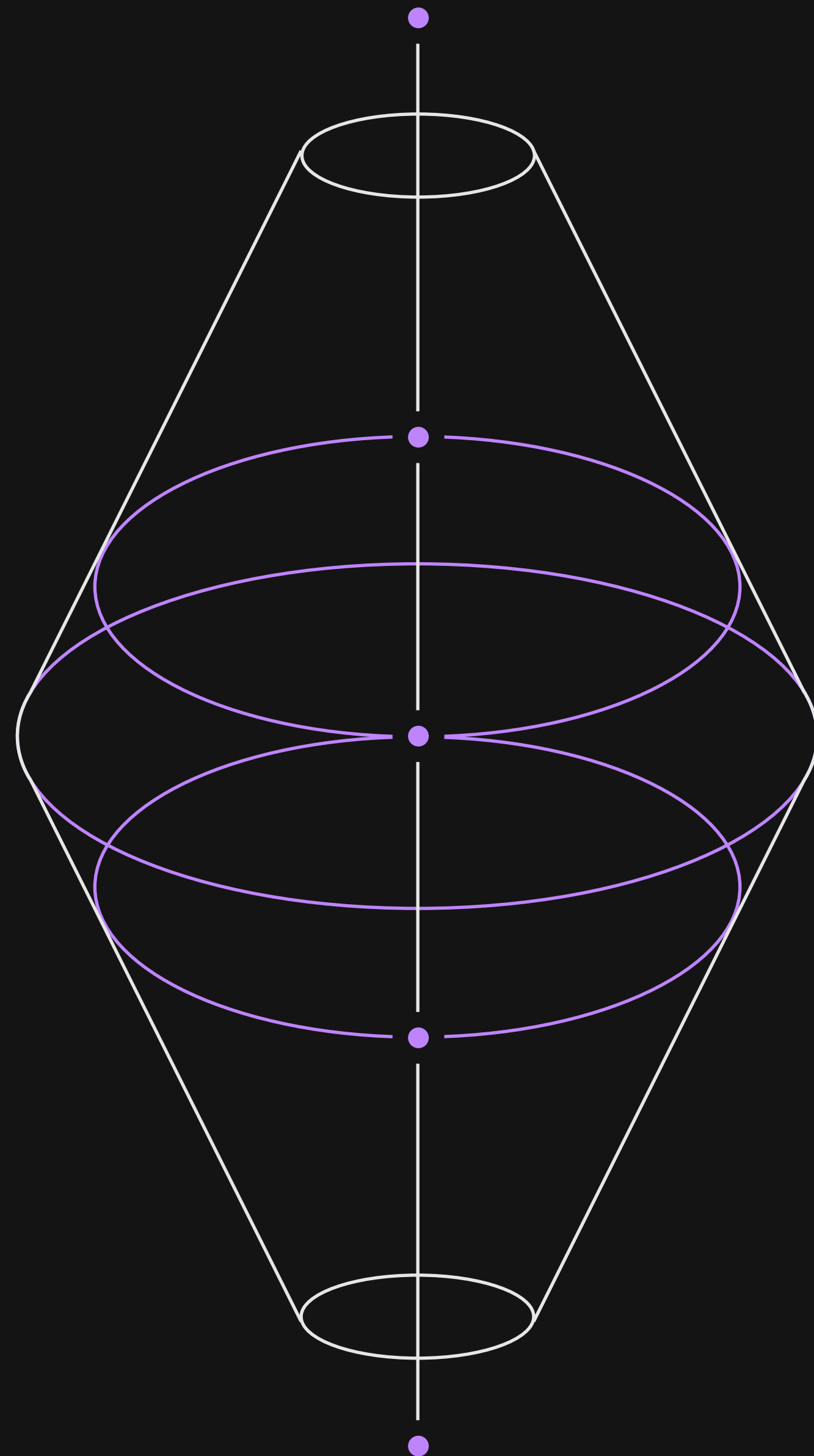
# Стереотипы @Component



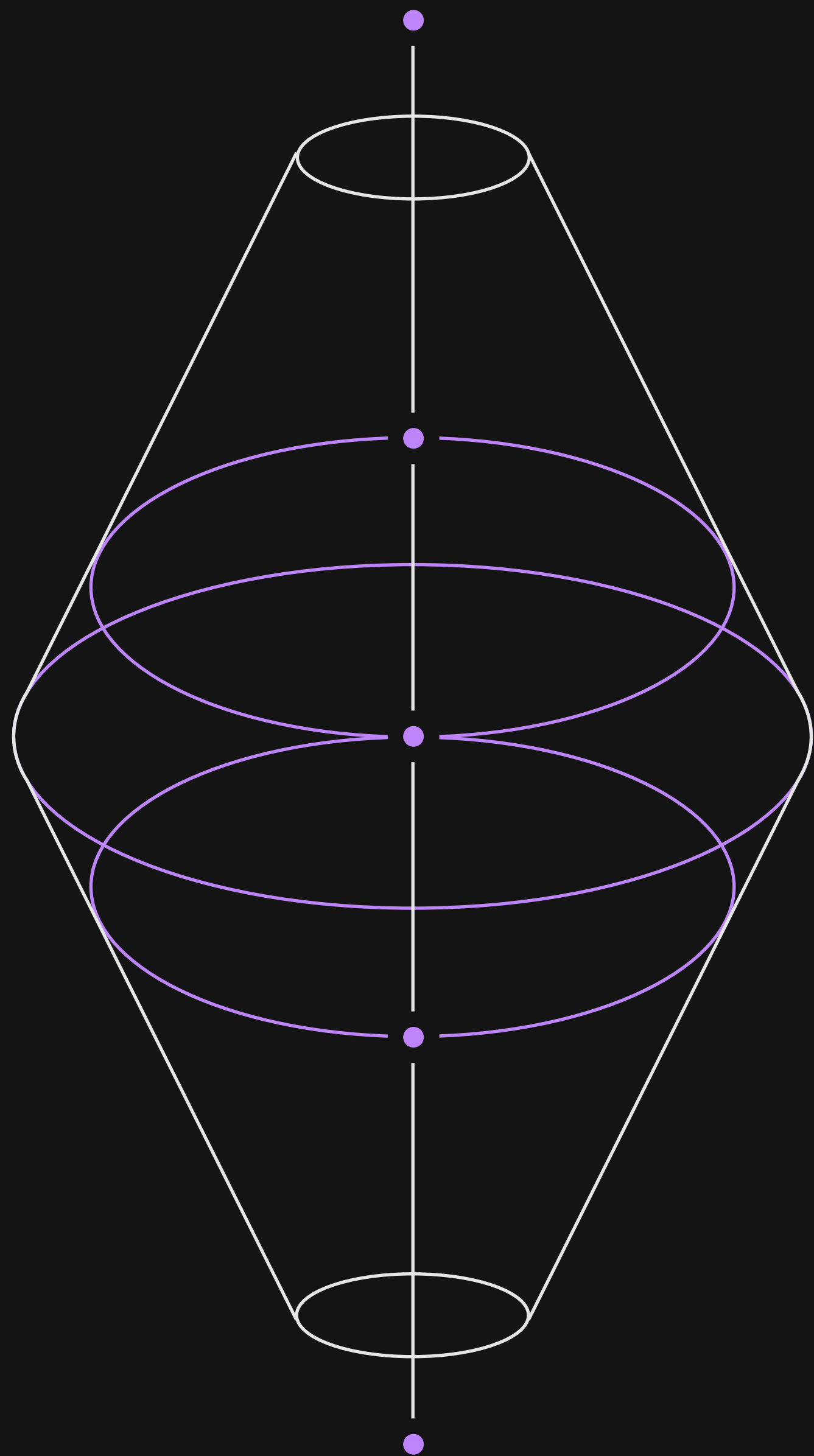
- @Service — бизнес-логика
- @Repository — доступ к данным
- @Controller — веб-слой MVC
- @RestController — REST API



# Инструменты: IntelliJ IDEA



- Вкладка Spring в Run/Debug
- Иконки на полях
- Навигация Related Beans
- Диаграммы зависимостей



# Плагины для Idea



- Spring Debugger
- Spring Explyt
- Spring Plugin & Spring Boot Plugin
- JPA Buddy
- Spring Assistant
- SonarLint





## Всегда ли нужен Spring?

Для tiny-проектов можно без него, но Spring Boot упрощает даже small-проекты.



## Field vs Constructor Injection?

Constructor Injection — best practice.

Причины: неизменяемость, тестируемость, гарантия инициализации.



## Spring не видит @Component?

**Проверьте следующее.**

1. Класс в зоне @ComponentScan
2. Есть аннотация @Component
3. Класс не абстрактный
4. Нет excludeFilters

# FAQ



## ? @Bean vs @Component?

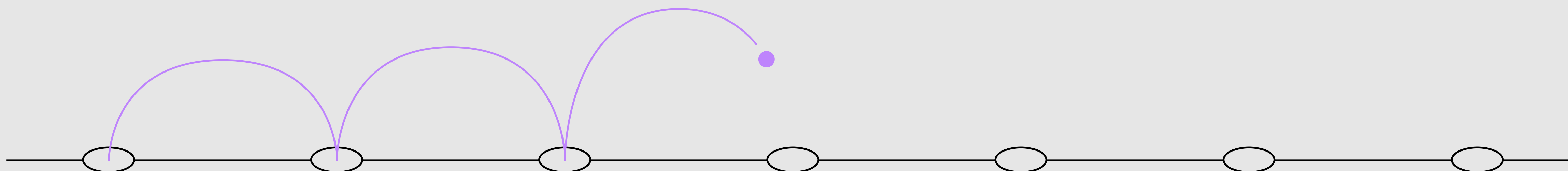
- @Component — для ваших классов
- @Bean — для настройки сторонних библиотек

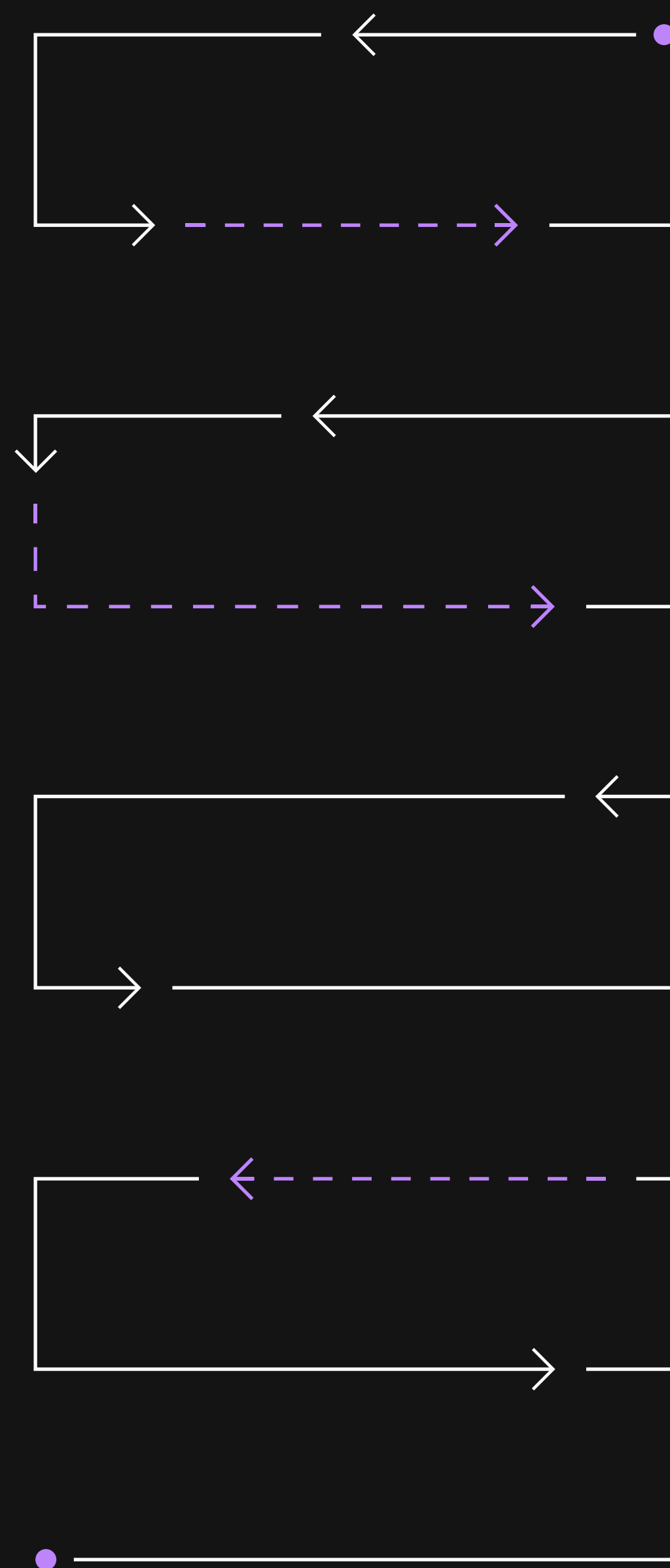
## ? Опциональные зависимости?

Используйте Optional<T>:  
@Autowired private  
Optional<CacheManager>  
cacheManager;

## ? @Qualifier vs @Primary?

- @Primary — глобальный выбор по умолчанию
- @Qualifier — явное указание в конкретном месте





# Ключевые выводы



- Spring навязывает хорошие практики (DI и IoC).
- Модульный Spring — под все аспекты разработки.
- В Spring есть много устоявшихся практик и паттернов.
- Constructor Injection — стандарт для обязательных зависимостей.
- @Qualifier и @Primary решают проблему неоднозначности.
- Понимание работы контекста — ключ к отладке.





ЦЕНТРАЛЬНЫЙ  
УНИВЕРСИТЕТ

# Вопросы

